

Data Warehouse Analyst

Вопросы оптимизации
производительности



Проверить, идет ли запись

Меня хорошо видно && слышно?



Ставим “+”, если все хорошо
“-”, если есть проблемы

Тема вебинара

Выгрузка данных из внешних систем



Борзилов Анатолий Сергеевич

Ведущий разработчик ООО «Делотех»

Об опыте (например):

Более 6-ти лет работаю в сфере Big Data

Телеграмм: @Borzilov_AS

Правила вебинара



Активно
участвуем



Off-topic обсуждаем
в Telegram **@OTUS DWH-2023-05**



Задаем вопрос
в чат или голосом



Вопросы вижу в чате,
могу ответить не сразу

Условные обозначения



Индивидуально



Время, необходимое
на активность



Пишем в чат



Говорим голосом



Документ



Ответьте себе или
задайте вопрос

Карта курса



Цели вебинара

К концу занятия вы сможете

1. Получить представление о проблемах с производительностью
2. Познакомиться с подходами и практиками в оптимизации
3. Изучить ключевые принципы оптимизации
4. Провести демонстрацию примеров из реальной жизни

Проблемы с производительностью: что это?

Проблемы с производительностью

- Долго ждем (время ожидания - latency)
- Потребляем много ресурсов (облачные бд)
- И то, и другое

Всё упирается в ресурсы – какие ресурсы?

- CPU (Вычислительные мощности)
- Disk (RAM)
- Network traffic

Задача оптимизации – получить желаемый результат (ответ на запрос) с минимумом затрат. То есть минимум CPU cycles, Table scans, Network shuffle (перераспределения данных по сети)

Compressing data

Encoding type	Keyword in CREATE TABLE and ALTER TABLE
Raw (no compression)	RAW
AZ64	AZ64
Byte dictionary	BYTEDICT
Delta	DELTA DELTA32K
LZO	LZO
Mostlyn	MOSTLY8 MOSTLY16 MOSTLY32
Run-length	RUNLENGTH
Text	TEXT255 TEXT32K
Zstandard	ZSTD

Сжатие данных

Методы:

- Использование SQL Server Management Studio (использование Transact SQL Command)
- Сжатие на уровне строк

```
ALTER TABLE tableName REBUILD WITH (DATA_COMPRESSION=ROW)
```

- Сжатие на уровне страниц

```
ALTER TABLE tableName REBUILD WITH (DATA_COMPRESSION=PAGE)
```

Сжатие данных

Общие	
Пространство данных	5,164 MB
Включен формат хранения Vardecimal	False
Место, занимаемое индексом	0,008 MB
Число строк	14262
Сжатие	
Тип сжатия	Row
Файловые группы	
Файловая группа FILESTREAM	
Таблица секционирована	False
Текстовая файловая группа	
Файловая группа	PRIMARY

Общие	
Пространство данных	9,734 MB
Включен формат хранения Vardecimal	False
Место, занимаемое индексом	0,008 MB
Число строк	14262
Сжатие	
Тип сжатия	None
Файловые группы	
Файловая группа FILESTREAM	
Таблица секционирована	False
Текстовая файловая группа	
Файловая группа	PRIMARY

Общие	
Пространство данных	2,008 MB
Включен формат хранения Vardecimal	False
Место, занимаемое индексом	0,008 MB
Число строк	14262
Сжатие	
Тип сжатия	Page
Файловые группы	
Файловая группа FILESTREAM	
Таблица секционирована	False
Текстовая файловая группа	
Файловая группа	PRIMARY

Негативные факторы сжатия данных

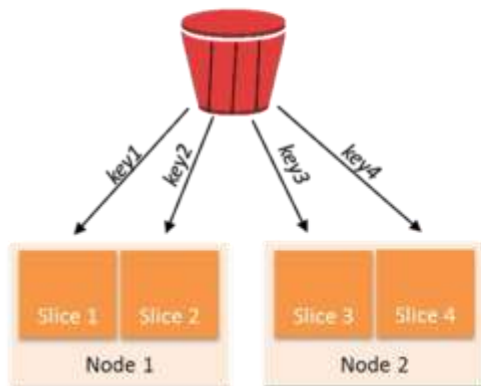
- Фрагментация таблиц
- Размер базы данных никаким образом нельзя сделать меньше, чем минимальный размер этой БД
- Увеличение нагрузки процессора

Physical data design

Rows distribution

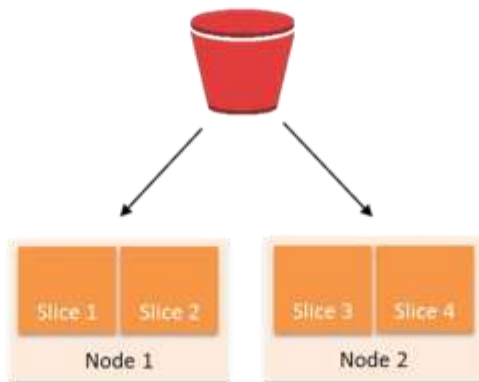
Distribution Key

key value to same location



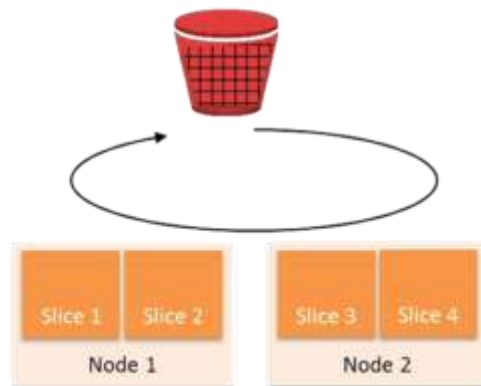
All

All data on every node

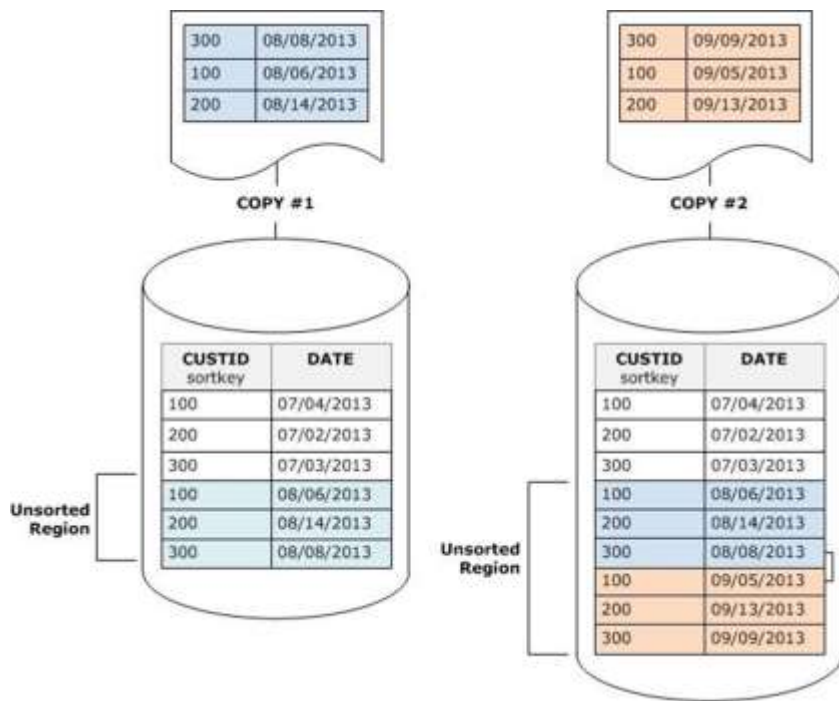


Even

Round robin distribution



Rows sort order



```
ENGINE = MergeTree
PARTITION BY toYYYYMM(date_in_poo)
ORDER BY office_id
SETTINGS index_granularity = 8192;
```


**Что вы слышали об
инкерментальном
обновлении?**

Способы обновления данных

- Инкрементально
- По периодам
- Полное пересоздание

```
insert into our_buffer_table
select new_data
from master_table;

truncate table our_table;
insert into our_table
select * from our_buffer_table;

truncate table our_buffer_table;
```

```
insert into our_table
select *
from table_with_data
where id > (
    select max(id)
    from table_cutoff
    where table_name = 'table_to_update'
)
```

dbt – incremental materialization

```
Artemiy Kzr, a week ago | 1 author (Artemiy Kzr)
1  {{
2    config (
3      materialized='incremental',      Artemiy Kzr, a week ago • Leverage SUPER data type, MATERIALIZED VIEWS; mod...
4      dist='request_id',
5      sort='request_id',
6      unique_key='request_id'
7    )
8  }}
9
10
11  -- noqa: disable=PRS,L028,L026
12  select
13
14    _id as request_id
15    , city_id
16
17    -- json fields to expand
18    , json_parse(price_details) as price_details
19    , json_parse(events) as events
20    , json_parse(log) as log
21
22    -- metadata
23    , {{ epoch_to_timestamp('__hevo__ingested_at') }} as __metadata_timestamp
24
25  from {{ source('wheely_prod', 'orders') }}
26  {{ hevo_filter_rows(last_n_days_of_data=true) }}
27  {% if is_incremental() %}      Artemiy Kzr, a week ago • Leverage SUPER data type, MATERIALIZED VIEWS; mod...
28    and request_id >= (select max(request_id) from {{ this }})
29  {% endif %}
```

dbt – insert_by_period

Usage:

```
{{
  config(
    materialized = "insert_by_period",
    period = "day",
    timestamp_field = "created_at",
    start_date = "2018-01-01",
    stop_date = "2018-06-01")
}}

with events as (

  select *
  from {{ ref('events') }}
  where __PERIOD_FILTER__ — This will be replaced with a filter in the materialization code

)

....complex aggregates here....
```

Configuration values:

- `period` : period to break the model into, must be a valid `datepart` (default="Week")
- `timestamp_field` : the column name of the timestamp field that will be used to break the model into smaller queries
- `start_date` : literal date or timestamp - generally choose a date that is earlier than the start of your data
- `stop_date` : literal date or timestamp (default=current_timestamp)

Способы обновления данных

Apache Airflow interface showing a list of DAGs (Directed Acyclic Graphs).

Navigation: DAGs, Data Profiling, Browse, Admin, Docs. Time: 15:30 MSK.

Search:

	DAG	Schedule	Owner	Recent Statuses	Links
1	On oda_load_gc_event_microbatches	* * * * *		2	
2	On oda_load_gc_event_lo_hdp	@daily		3	
3	On oda_load_gwt	@daily		3	
4	On oda_load_hawk_new	1 day, 0:00:00		8	
5	On oda_load_hc_dicts	1 day, 0:00:00		8	
6	On oda_load_hc_pg_juc5	1 day, 0:00:00		8	
7	On oda_load_its	@hourly		8	
8	On oda_load_jh	1 day, 0:00:00		10	
9	On oda_load_jw	1 day, 0:00:00		10	
10	On oda_load_large_sf_ps4	1 day, 0:00:00		10	

Showing 31 to 40 of 99 entries

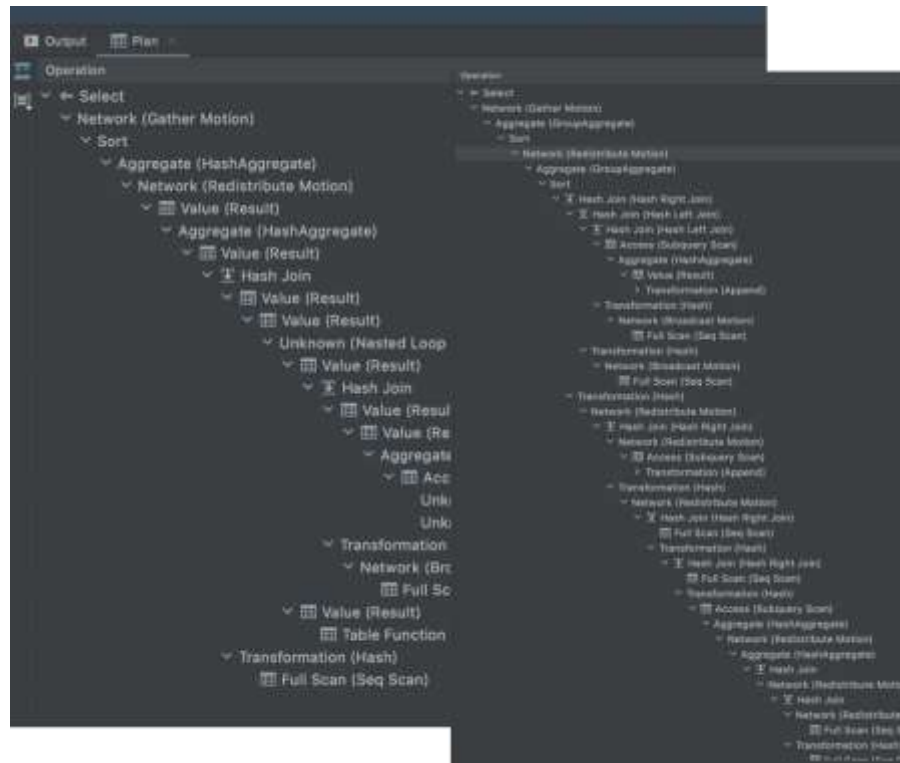
Previous 1 2 3 4 5 10 Next

**Как будет выполнен
запрос?**

EXPLAIN – анализ плана запроса

Основные операции:

- Seq Scan
- Index Scan
- Nested Loops
- Hash Join
- Sort
- Aggregate
- Unique



Остальные операции:

- Bitmap Index Scan
- Merge Join
- GroupAggregate
- HashAggregate
- Filter
- Limit
- Append
- HashSetOp
- Materialize
- CTE Scan
- SubPlan
- InitPlan
- Subquery Scan

Форматы плана запроса

```
← Select
  Network (Gather Motion)
    Value (Result)
      Aggregate (GroupAggregate)
        Sort
          Network (Redistribute)
            Value (Result)
              Aggregate (Hash)
                Full Scan ( table: v_late_rids_duration_in_poo;
```

```
74
75 Planning time: 160.226 ms
76 (slice0) Executor memory: 970K bytes.
77 (slice1) Executor memory: 18120K bytes avg x 72 workers, 18120K ...
78 (slice2) Executor memory: 60K bytes avg x 72 workers, 60K bytes ...
79 (slice3) Executor memory: 19592K bytes avg x 72 workers, 19592K ...
80 * (slice4) Executor memory: 10879K bytes avg x 72 workers, 193040K...
81 (slice5) Executor memory: 138K bytes avg x 72 workers, 240K byte...
82 Memory used: 589824kB
83 Memory wanted: 861599kB
84 Optimizer: Postgres query optimizer
85 Execution time: 80208.099 ms
```

```
QUERY PLAN
1 Gather Motion 72:1 (slice1; segments: 72) (cost=428761756.78..4335553_
2 -> GroupAggregate (cost=428761756.78..433555348.41 rows=887703 width_
3 Group Key: ((sh.close_dt)::date), sh.src_office_id, bo1.office_n_
4 -> Sort (cost=428761756.78..428921543.17 rows=887703 width=106_
5 Sort Key: ((sh.close_dt)::date), sh.src_office_id, bo1.off_
6 -> Redistribute Motion 72:72 (slice10; segments: 72) (c_
7 Hash Key: ((sh.close_dt)::date), sh.src_office_id, b_
8 -> GroupAggregate (cost=305426919.37..309102006.30_
9 Group Key: ((sh.close_dt)::date), sh.src_offic_
10 -> Sort (cost=305426919.37..305506705.76 row_
Sort Key: ((sh.close_dt)::date), sh.src_
-> Hash Right Join (cost=222330841.31..
Hash Cond: (pc.rid = td.rid)
-> Hash Left Join (cost=11440744_
Hash Cond: (pc.create_dt = m_
-> Hash Left Join (cost=11_
Hash Cond: ((CASE WHEN_
-> Subquery Scan on p_
-> HashAggregat_
Group Key:_
-> Result_
-> _
```


Практики оптимизации производительности

Best practices to write high performing SQL queries

- Используйте индексы
- Не используйте циклы с большим количеством итераций
- Избегайте коррелирующих подзапросов
- Старайтесь не использовать `SELECT *`
- Пользуйтесь временными таблицами с умом
- Используйте `EXISTS()`
- Всегда пишите `alias` для своих таблиц

Не используйте циклы с большим количеством итераций

```
for (int i = 0; i < 1000; i++)  
{  
    SqlCommand cmd = new SqlCommand(«INSERT INTO TBL (A,B,C) VALUES...»);  
    cmd.ExecuteNonQuery();  
}
```

```
UPDATE TableName  
SET A = @VALUE  
WHERE B = 'YOUR CONDITION'  
AND A <> @VALUE — VALIDATION
```

```
INSERT INTO TableName (A,B,C) VALUES (1,2,3),(4,5,6),(7,8,9)  
  
UPDATE TableName SET A = CASE B  
    WHEN 1 THEN 'NEW VALUE'  
    WHEN 2 THEN 'NEW VALUE 2'  
    WHEN 3 THEN 'NEW VALUE 3'  
END  
WHERE B in (1,2,3)
```

Избегайте коррелирующих подзапросов

```
SELECT c.Name,  
       c.City,  
       (SELECT CompanyName FROM Company WHERE ID = c.CompanyID) AS CompanyName  
FROM Customer c
```

```
SELECT c.Name,  
       c.City,  
       co.CompanyName  
FROM Customer c  
LEFT JOIN Company co ON c.CompanyID = co.CompanyID
```

Пользуйтесь временными таблицами с умом

```
SELECT * INTO #Temp FROM Customer WHERE RegionID = 5
```

```
SELECT r.RegionName, t.Name FROM Region r JOIN #Temp t ON t.RegionID = r.RegionID
```

```
SELECT r.RegionName, t.Name FROM Region r  
JOIN (SELECT Name, RegionID FROM Customer WHERE RegionID = 5) AS t  
ON t.RegionID = r.RegionID
```

Используйте EXISTS()

```
IF (SELECT COUNT(1) FROM EMPLOYEES WHERE FIRSTNAME LIKE '%JOHN%') > 0  
    PRINT 'YES'
```

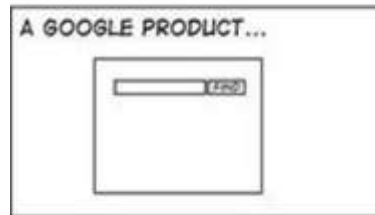
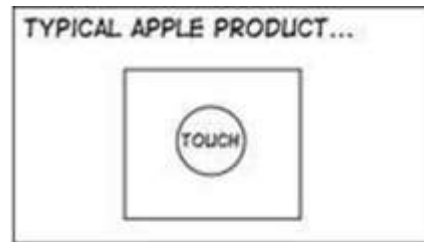
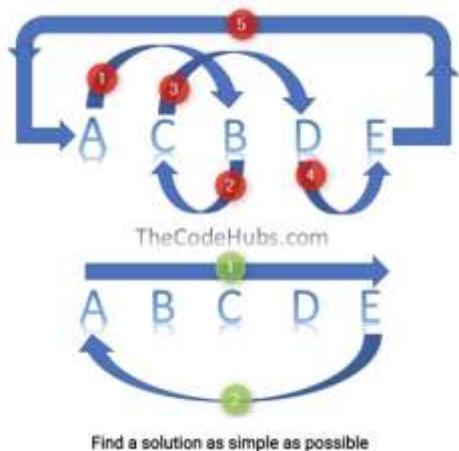
или

```
IF EXISTS(SELECT FIRSTNAME FROM EMPLOYEES WHERE FIRSTNAME LIKE '%JOHN%')  
    PRINT 'YES'
```

Code refactoring & KISS

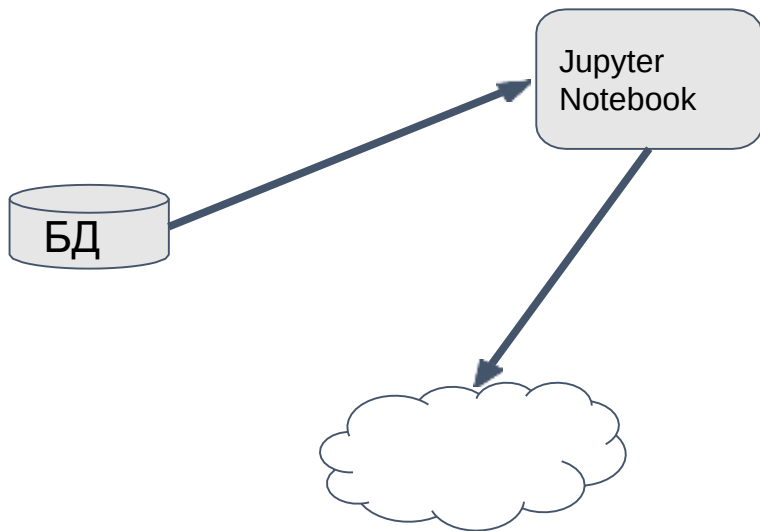
Что такое KISS?)

Это означает, что для любого запроса или задачи следует использовать самое простое, наиболее управляемое и одно из лучших решений



Keep it simple, stupid

Задача: рассчитать «Мотивацию на ПВЗ»



Проблемы:

1. Изначально неоптимизированный код
2. Долгий и тяжелый запрос к БД и получение значений
3. Отсутствие автоматизации
4. ...



Решения - ?

Code refactoring

Цели:

- Оптимизация времени исполнения запроса
- Оптимизация используемой для запроса памяти
- Улучшение дизайна запроса

Code refactoring

Признаки необходимости модификации запроса:

- Плохое оформление кода
- Наличие дублирующего кода
- Невозможность визуального представления запроса
- Наличие в запросе медленно работающих конструкций
- Невозможность построения на основе запроса индексированного представления
- ...

Code refactoring

Решения:

- Выделение пользовательской функции
- Выделение представления
- Избавление от подзапросов
- Избавление от Union
- ...

Пример оптимизации запроса БД

Оптимизация запросов базы данных на примере B2B сервиса для строителей

Исходные данные: сервис для управления бизнес процессами в строительных компаниях, с которым работает около 3000 компаний

PostgreSQL 9.6. В базе данных около 300 таблиц и каждые сутки в нее поступает до 200 млн запросов (в среднем, 3-4 тыс. запросов в секунду)

Источники проблем:

1. Работа на скорую руку
2. Решение проблем с производительностью индексами

Выделение типов запросов:

1. “Топы”
2. “Середняки”
3. “Хвост”

Типы запросов

Топы:

- Очень часто вызываются либо медленно выполняются
- Их немного
- Оптимизация может освободить 5-10% ресурсов

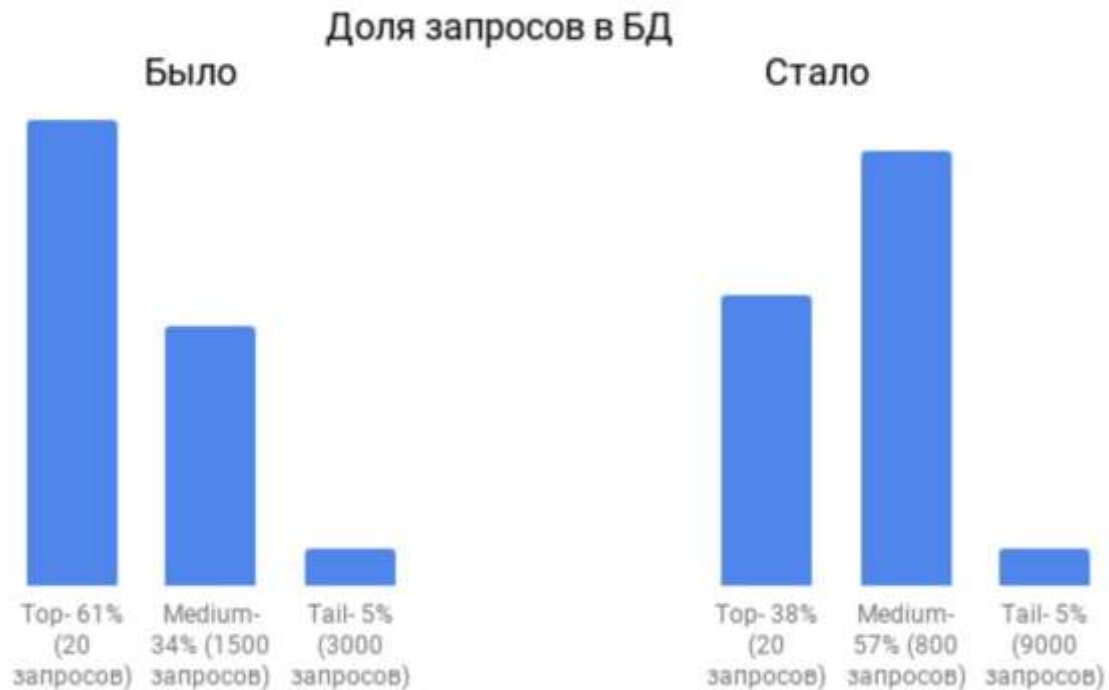
Середняки:

- Все запросы, кроме Топов и последних 5-10%
- Эти запросы могут «весить» до 80%
- Их оптимизация может сильно поднять производительность сервера

Хвост

```
1 SELECT sum(time_top) AS sum_top,
2       sum(time_medium) AS sum_medium,
3       sum(time_tail) AS sum_tail
4 FROM (
5       SELECT CASE
6             WHEN rn <= 20 THEN tt_percent
7             ELSE 0
8       END AS time_top,
9       CASE
10            WHEN rn > 20
11              AND rn <= 800 THEN tt_percent
12            ELSE 0
13       END AS time_medium,
14       CASE
15            WHEN rn > 800 THEN tt_percent
16            ELSE 0
17       END AS time_tail
18     FROM (
19           SELECT total_time / (
20                 SELECT sum(total_time)
21                 FROM pg_stat_statements
22                 ) * 100 AS tt_percent,
23           query,
24           ROW_NUMBER () OVER (
25                 ORDER BY total_time DESC
26           ) AS rn
27     FROM pg_stat_statements
28     ORDER BY total_time DESC
29   ) AS t
30 ) AS ts
```

Типы запросов



Используемые приемы оптимизации для Топов

- Redesign системы
- Добавление или изменение индексов
- Переписывание ORM запросов на чистый SQL
- Переписывание логики ленивой подгрузки данных
- Кеширование через денормализацию данных
- Кеширование статических таблиц со справочниками
меняющихся таблиц в памяти программы

Используемые приемы оптимизации для Средняков

- Использование EXISTS вместо COUNT
- Избавление от DISTINCT
- Отказ от COUNT в пользу расчета приближенного значения

```
SELECT DISTINCT P.ID, P.FIRST_NAME, P.LAST_NAME
FROM DELIVERY D JOIN PERSON P ON D.DRIVER_ID = P.ID;

SELECT P.ID, P.FIRST_NAME, P.LAST_NAME
FROM PERSON
WHERE EXISTS(SELECT D.ID FROM DELIVERY WHERE D.DRIVER_ID = P.ID);|
```

Мораль

Что изучить еще?

- Using Advisors
- Vectorized functions
- Materialized views with incremental update
- Run on schedule: VACUUM + ANALYZE
- Handling bursts of workload with concurrency scaling and elastic resize
- Taking advantage of Data Lake integration
- Query queues: priorities to increase throughput
- Leverage efficiency of temporary tables
- Maintaining efficient data loads (parallel)

Список материалов для изучения

1. <https://www.interaction-design.org/literature/article/kiss-keep-it-simple-stupid-a-design-principle>
2. [Вопросы оптимизации](#)
3. [Explain PostgreSQL](#)

Вопросы?



Ставим “+”,
если вопросы есть



Ставим “-”,
если вопросов нет

Рефлексия

Цели вебинара

Проверка достижения целей

1. Получить представление о проблемах с производительностью
2. Познакомиться с подходами и практиками в оптимизации
3. Изучить ключевые принципы оптимизации
4. Провести демонстрацию примеров из реальной жизни

**Заполните, пожалуйста,
опрос о занятии
по ссылке в чате**

Спасибо за внимание!